PROJECT KALKI ASSISTANT CORE

Autonomous Multi-Lingual Desktop AI Assistant (J.A.R.V.I.S. Core Architecture)

1. Executive Overview & Mission

Project KALKI is a production-grade, autonomous desktop assistant designed to deliver low-latency OS automation, intelligent natural language reasoning, computer vision analysis, and multi-lingual voice feedback. Engineered with a decoupled architecture, it pairs a native **Electron + React** user dashboard with an asynchronous **FastAPI Python Core Engine**. KALKI achieves 0ms local command execution for standard applications and system hotkeys while seamlessly leveraging Google GenAI's gemini-3.8-flash for complex multi-lingual dialogues.

2. System Architecture & Tech Stack

Tier / Layer	Technology	Function & Responsibility
Desktop Frontend	Electron 29, React 18, Vite 5	Hardware-accelerated HUD interface, real-time audio capture, desktop screen streaming, macro management, and telemetry views.
Backend Gateway	FastAPI, Uvicorn, AsyncIO	Asynchronous REST API endpoints (/execute-command, /listen-voice, /process-frame, /macros). Non-blocking background worker dispatch.
Intelligence Engine	Dual-Tier: Local Router + Google GenAI	Tier 1 executes local regex OS commands at 0ms. Tier 2 routes non-local prompts to gemini-3.8-flash with explicit 10s deadlines.
Voice & Speech (TTS/STT)	Edge-TTS, SpeechRecognition, Windows Media	Kannada, Hindi, and Indian English script detection; native Windows MediaPlayer audio playback.
Computer Vision	OpenCV 5, NumPy	Adaptive thresholding, contour extraction, spatial clustering to identify active UI widgets and bounding boxes from display streams.
Persistence & Telemetry	PostgreSQL, SQLAlchemy ORM	User configuration, custom voice macro triggers, and complete execution audit logging with automated failover.

3. Core Subsystems & Operational Flow

A. Hybrid Dual-Tier Intelligence Engine (llm_engine.py)

KALKI eliminates cloud latency for known system commands by employing a two-tier intent routing pipeline:

- **Tier 1 (Fast-Path Local Router):** Matches regex patterns for application launching (Chrome, VS Code, Calculator, Notepad, Windows Terminal) and PyAutoGUI keyboard typing (type <text>) with 0ms cloud overhead.
- **Tier 2 (Cloud LLM Core):** Directs general queries to Google GenAI's gemini-3.8-flash. Governed by a multilingual J.A.R.V.I.S. system prompt that preserves conversational tone across English, Kannada, Hindi, and Kanglish.
- **Fault-Tolerant Timeout Safeguard:** Implements an explicit 10,000ms deadline (types.HttpOptions(timeout=10000)) wrapped in try-except blocks, preventing network deadlocks and ensuring instantaneous recovery.

B. Multilingual Speech & Voice Engine (tts_engine.py)

• **Speech Recognition:** Records 16 kHz 1-channel audio via sounddevice and decodes user voice using Google Speech API with automatic Kannada (kn-IN) and English (en-IN) fallbacks.
• **Neural Voice Synthesis:** Automatically inspects Unicode ranges to assign voices (Kannada: *kn-IN-GaganNeural*, Hindi: *hi-IN-MadhurNeural*, English: *en-IN-PrabhatNeural*).
• **Decoupled Playback:** Generated MP3 streams play asynchronously through Windows PowerShell System.Windows.Media.MediaPlayer in non-blocking thread workers.

C. Computer Vision Pipeline (/process-frame)

Accepts base64-encoded frame snapshots from the Electron screen capture stream. Applies Gaussian blur, adaptive Gaussian thresholding, and contour tree parsing to identify desktop UI controls, windows, and buttons, returning structured spatial coordinates and bounding box metrics for contextual awareness.

4. Key Milestones & Engineering Resolutions

Problem / Vulnerability	Root Cause	Architectural Solution Implemented
UI Freezing on 'KALKI: Thinking...'	Unbounded synchronous network call inside FastAPI async def route without explicit timeout.	Enforced types.HttpOptions(timeout=10000), offloaded intent processing to asyncio.to_thread(), and wrapped in graceful error handling.
Google GenAI 400 INVALID_ARGUMENT	GenAI client SDK enforces a strict minimum deadline of ≥ 10s. Sub-10s timeouts are rejected.	Calibrated timeout to exactly 10,000ms, adhering strictly to Google GenAI server specifications.
HTTP Response Delayed by TTS	Speech synthesis was executed synchronously in request-response lifecycle.	Decoupled TTS using asyncio.create_task(speak_text(...)). FastAPI immediately delivers JSON to Electron while audio plays concurrently.
PostgreSQL Crash Vulnerability	Schema generation crashed application startup if PostgreSQL was offline.	Added connection timeout guards (connect_timeout=2) and resilient fallback wrappers around DB queries and audit logs.

5. Project Repository Layout

```
ProjectKalki/ ■■■ main.js # Electron main lifecycle & backend process spawner ■■■ preload.js # Electron IPC context isolation bridge ■■■ package.json # Electron/React lifecycle dependencies & scripts ■■■ vite.config.js # Vite build & bundle pipeline configuration ■■■ src/ ■ ■■■ App.jsx # React HUD interface, voice controls, macro table ■ ■■■ main.jsx # React root application bootstrap ■ ■■■ index.css # Global HUD styling ■■■ backend/ ■■■ .env # Local credentials (GEMINI_API_KEY, GROQ_API_KEY) ■■■ app.py # FastAPI application server & REST route handlers ■■■ llm_engine.py # Dual-tier router & Gemini 3.8 Flash intelligence core ■■■ tts_engine.py # Multilingual Edge-TTS speech synthesizer ■■■ database.py # SQLAlchemy engine with connection timeout guards ■■■ models.py # Relational schema (User, VoiceMacro, AuditLog) ■■■ requirements.txt # Python runtime requirements ■■■ test_diagnostics.py # Automated end-to-end verification test suite
```

6. Diagnostic Verification Status

All critical paths have been verified via backend/test_diagnostics.py:

- **Check 1 (.env Resolution):** PASS — Explicitly loads from backend/.env via Path(__file__).resolve().parent / '.env'.
- **Check 2 (Model Verification):** PASS — Confirmed gemini-3.8-flash is active with google-genai.
- **Check 3 (Timeout Handling):** PASS — Validated 10s deadline with clean exception trapping.
- **Check 4 (Decoupled TTS):** PASS — Verified non-blocking background dispatch via asyncio.create_task().
- **Check 5 (Endpoint Execution):** PASS — Confirmed POST /execute-command returns immediate JSON status.